

Day 1 종합 실습 실행결과보고서

실무형 수집·검증·품질 파이프라인

작성자	조현강 (광주 4반)
실습명	Day 1 종합 실습 - 실무형 수집·검증·품질 파이프라인
사용 API	Open-Meteo, Countries.dev, ip-api
실행 환경	Python 3.11, venv, httpx, pydantic v2, pandas, pyarrow, pytest, ruff

1. 실행 결과 요약

3개 공개 API를 `asyncio.gather()`로 동시에 수집했고, 수집한 JSON에서 필요한 필드를 추출하여 Pydantic v2 모델로 타입과 범위를 검증했습니다. 검증 통과 데이터 74건을 CSV와 Parquet 두 형식으로 저장한 뒤, 각 형식의 읽기/쓰기 시간을 측정해 비교했습니다.

항목	결과
API 수집	ip-api, countries_dev, open_meteo 모두 성공
Pydantic 검증	성공 74건 / 실패 0건
산출물	output/valid_records.csv, output/valid_records.parquet
pytest	6 passed
ruff	All checks passed!

2. 실행결과 화면 Capture

캡처 1. 메인 파이프라인 실행 결과

```
(.venv) hk@Jayui-MacBookPro day1-pipeline-main % python main.py 2>/dev/null
[수집 성공] ip_api
[수집 성공] countries_dev
[수집 성공] open_meteo

== Pydantic 검증 결과 ==
검증 성공 : 74건
검증 실패 : 0건

== CSV / Parquet 저장 및 성능 비교 ==
CSV 쓰기 (저장) 시간 : 0.005957초
CSV 읽기 시간 : 0.001577초
CSV 파일 크기 : 3.79KB
Parquet 쓰기 (저장) 시간 : 1.626349초
Parquet 읽기 시간 : 2.856105초
Parquet 파일 크기 : 9.96KB
쓰기 (저장)가 더 빠른 형식 : CSV
읽기가 더 빠른 형식 : CSV
저장된 행 수 : 74건
```

캡처 2. CSV / Parquet 산출물 생성 확인

```
○ (.venv) hk@Jayui-MacBookPro day1-pipeline-main %  
○ (.venv) hk@Jayui-MacBookPro day1-pipeline-main %  
● (.venv) hk@Jayui-MacBookPro day1-pipeline-main % ls -lh output  
total 32  
-rw-rw-r--@ 1 hk  staff   3.8K Jul 21 00:24 valid_records.csv  
-rw-rw-r--@ 1 hk  staff  10K Jul 21 00:24 valid_records.parquet
```

캡처 3. pytest 스키마 검증 테스트 결과

```
(.venv) hk@Jayui-MacBookPro day1-pipeline-main % pytest  
===== test session starts =====  
platform darwin -- Python 3.11.1, pytest-8.4.1, pluggy-1.6.0  
rootdir: /Users/hk/Downloads/day1-pipeline-main  
plugins: anyio-4.14.2  
collected 6 items  
  
tests/test_pipeline.py ..... [100%]  
  
===== 6 passed in 1.32s =====
```

캡처 4. ruff 코드 스타일 검사 결과

```
● (.venv) hk@Jayui-MacBookPro day1-pipeline-main % ruff check .  
All checks passed!  
○ (.venv) hk@Jayui-MacBookPro day1-pipeline-main %
```

3. Code 분석 결과에 대한 본인 의견

이번 코드는 수집, 검증, 저장, 출력 흐름을 함수별로 나누어 작성했습니다. `main.py`는 전체 실행 흐름을 담당하고, 실제 기능은 `src/pipeline.py`에 분리했습니다. 이렇게 나누면 실행 파일은 단순해지고, 수집 함수나 검증 함수는 테스트하기 쉬워집니다.

API 수집에는 `asyncio.gather()`를 사용했습니다. 3개 API를 순서대로 하나씩 호출하면 앞 API가 끝날 때까지 다음 API가 기다려야 하지만, `asyncio.gather()`를 사용하면 Open-Meteo, Countries.dev, ip-api 요청을 동시에 시작할 수 있습니다. 그래서 네트워크 대기 시간을 줄일 수 있습니다.

스키마 검증에는 Pydantic v2의 `BaseModel`, `Field`, `field_validator`를 사용했습니다. 기온, 강수량, 위도, 경도처럼 범위가 중요한 값은 `Field`로 제한했고, 비어 있으면 안 되는 문자열은 `field_validator`로 검사했습니다. 검증 중 오류가 발생하면 `ValidationError`를 처리하여 성공 데이터와 실패 데이터를 분리하도록 만들었습니다.

저장 단계에서는 검증 통과 데이터만 CSV와 Parquet 두 형식으로 저장했습니다. `to_csv()`와 `to_parquet()` 실행 시간을 각각 쓰기 시간으로 측정했고, `read_csv()`와 `read_parquet()` 실행 시간을 각각 읽기 시간으로 측정했습니다. 이번 실행에서는 데이터 건수가 74건으로 작아서 CSV가 쓰기과 읽기 모두 더 빠르게 나왔습니다.

4. 추가 의견 및 개선 사항

- API 요청 실패 시 재시도 로직을 추가하면 네트워크 오류에 더 안정적으로 대응할 수 있습니다.
- 현재는 모든 검증 통과 데이터를 하나의 CSV와 Parquet 파일로 저장하지만, source별 파일로 나누면 분석이 더 쉬울 수 있습니다.
- 테스트는 스키마 검증 중심으로 작성되어 있으므로, 추후에는 API 응답 처리와 저장 성능 측정 함수 테스트도 추가할 수 있습니다.
- API URL, 좌표, IP 주소를 코드에 직접 적기보다 설정 파일이나 환경변수로 분리하면 재사용성이 좋아집니다.
- 데이터가 커질 경우 CSV와 Parquet의 성능 차이를 더 명확하게 확인할 수 있으므로, 대용량 샘플 데이터로도 실험해볼 수 있습니다.